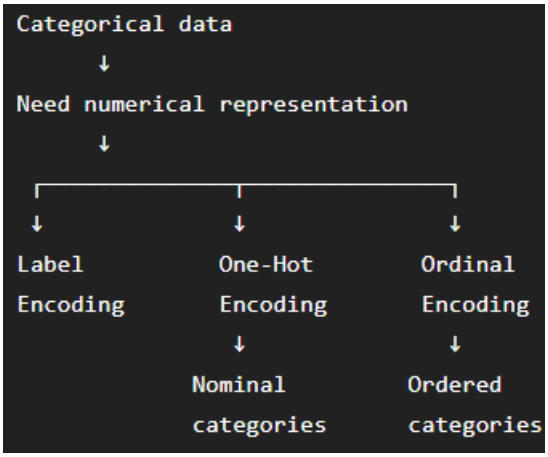
Lab 03

Code	Purpose
df.head()	First 5 rows
df.dtypes	Data types of columns
df.select_dtypes()	Select columns based on data type
df.isnull()	Identify missing values
df.isnull().sum()	Count missing values
df.duplicated()	Identify duplicate rows
df.drop_duplicates()	Remove duplicate rows
fillna()	Fill missing values
median()	Calculate median
mode()	Calculate most frequent value

age ↓ numerical ↓ median imputation	embarked / embark_town ↓ categorical ↓ mode imputation	deck ↓ many missing values ↓ "Unknown" category

Convert **categorical data into numerical representations**

Encoding	Appropriate situation
Label Encoding	Assign integer labels to categories
One-Hot Encoding	Nominal categories with no meaningful order
Ordinal Encoding	Categories have a meaningful order



Quick memory points

- pd.get_dummies() → **One-Hot Encoding**
- **Nominal** → no meaningful order
- **Ordinal** → meaningful order
- Low < Medium < High → **Ordinal**
- One-hot → separate **0/1 columns**
- Arbitrary Red=0, Blue=1, Green=2 can create a **false ordering**

Method	Meaning
fit()	Learn from data
transform()	Apply learned transformation
fit_transform()	Learn + apply

Feature Scaling

StandardScaler ↓ mean ≈ 0 standard deviation ≈ 1	MinMaxScaler ↓ values generally scaled to [0, 1]

Scaler	Main idea
StandardScaler	Mean = 0, standard deviation = 1
MinMaxScaler	Usually scales to 0–1
RobustScaler	Uses **median and IQR**, more robust to outliers

IQR means:

> **Interquartile Range**

and is calculated as:

```
IQR = Q3 - Q1
```

where:

- `Q1` = first quartile
- `Q3` = third quartile

The IQR represents the spread of the **middle 50%** of the data.

Feature Transformation

Transformation	Effect on large values
Square Root **x' = √x**	Compresses
Log **x' = log(x)**	Compresses more strongly

Box-Cox/Power Transformation; $x' = \frac{x^{\lambda}-1}{\lambda}$

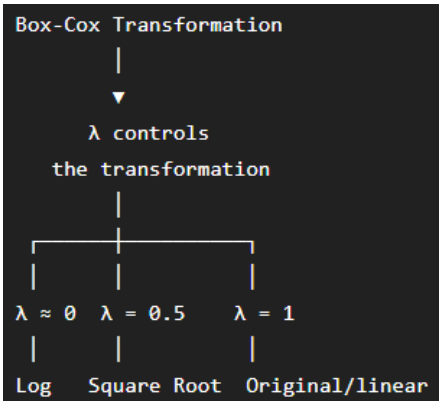
Different values of λproduce different transformations.

Important examples

When $\lambda = 1$ → The transformation is essentially a linear/no meaningful transformation of the original data.

When $\lambda = 0.5$ → It is related to a **square root transformation**. $x^{0.5} = \sqrt{x}$

When $\lambda \to 0$ → Box-Cox approaches a **log transformation**: $x' = \log(x)$



Transformation	Can handle 0?	Can handle negatives?
Square Root	Yes	No
np.log(x)	No	No
np.log1p(x)	Yes	No, if $x < -1$
Box-Cox	No, standard form requires $x > 0$	No

Lab 04

Relevant → useful for predicting the target **Irrelevant** → does not provide useful information **Redundant** → information is duplicated or can be derived from other features	**+1** → Perfect positive correlation **0** → No linear correlation **-1** → Perfect negative correlation

Concept	Answer
Function used to calculate correlation	.corr()
Visualization used	Heatmap
annot=True	Shows numerical values
fmt=".2f"	Shows 2 decimal places
sex relationship with survived	Strong negative correlation in this encoding
pclass relationship with survived	Negative correlation
Highly correlated pair	fare and fare_per_person
Correlation mentioned in notebook	0.84
Suggested feature to remove	fare

feature-selection approaches:

Method	Type
Correlation Analysis	**Filter Method**
Forward Feature Selection	**Wrapper Method**
Random Forest Feature Importance	**Embedded Method**

PCA
↓
Dimensionality Reduction

Before PCA ↓ Standardize features ↓ PCA	PCA(n_components=0.95) ↓ Retain at least 95% of variance
explained_variance_ratio_ ↓ Variance explained by each PC	np.cumsum(...) ↓ Cumulative explained variance

Oversampling
↓
Dataset size increases
↓
Duplicates minority samples
↓
Possible overfitting

Undersampling → Dataset size decreases → Removes majority samples → Possible information loss

Method	Dataset size	What happens?	Main disadvantage
Random Oversampling	Increases	Duplicates minority samples	Overfitting
Random Undersampling	Decreases	Removes majority samples	Information loss

Lab 05

Linear Score → Logistic Regression first calculates a linear equation: $z = \theta_0 + \theta_1 x$

Where: ϑ_0 → *intercept / bias* | ϑ_1 → *coefficient / weight* | x → *input feature* | z → *linear score*

Sigmoid Function → $\sigma(z) = \frac{1}{1+e^{-z}}$

Any real number → Sigmoid → A value between 0 and 1 → Probability

Log-Loss / Binary Cross-Entropy → $J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y_i \log(h_i) + (1-y_i)\log(1-h_i)]$

y → actual value | **h** → predicted probability | **m** → number of observations

Linear Regression → often uses Mean Squared Error	**Logistic Regression** → uses Log-Loss / Binary Cross-Entropy

Log-Loss heavily penalizes confident incorrect predictions; *Example:*

Actual = 1 | Predict 0.49 → somewhat wrong | Predict 0.001 → very confidently wrong → huge penalty

Gradient → Tells us how to update the parameters

Gradient Descent → **θ = θ - α × gradient**

The classification threshold converts:

Probability → Class decision

Learning Rate	Result
Too large	Overshooting / oscillation / divergence
Too small	Very slow convergence
Appropriate	Efficient convergence

k-Nearest Neighbors (kNN)

To classify a new data point, look at the **K nearest training data points** and let them vote.

For two points: (x_1, y_1) and (x_2, y_2)

the **Euclidean distance** is: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

In machine learning notation: $d(x, x') = \sqrt{\sum (x_i - x_i')^2}$

- A **smaller distance** means the two points are more similar/closer in the feature space.

Step 1 — **Calculate distance from the new point to every training point**

→ **Step 2** — **Sort the distances**

→ **Step 3** — **Select the K nearest neighbors**

↓

Step 4 — **Look at their classes**

→ **Step 5** — **Majority voting**

↓

Final prediction

Feature	Logistic Regression	kNN
Main use	Classification	Classification
Learns parameters	Yes	No explicit parameter learning in basic kNN
Uses Sigmoid	Yes	No
Uses distance	Not for classification decisions	Yes
Training process	Gradient Descent/optimizer	Stores training data
Prediction	Uses learned equation	Finds nearest neighbors
Scaling importance	Can be useful	Very important for distance calculations

Feature scaling → important because kNN uses distance

Small K	Large K
→ Complex	→ Smooth
→ Sensitive	→ Stable
→ Can overfit	→ Can underfit